

LARA: Learned Alignment with Replay Assurance

A Reusable Learned Candidate Generator with Exact Certification

Alessandro Berti Wil M.P. van der Aalst

Process and Data Science (PADS), RWTH Aachen University

ICPM 2027

The Problem

- **Alignments** explain, event by event, how a trace deviates from a process model
- Optimal alignments need **exact shortest-path search** (A^*): often the **bottleneck** on real discovered models
- Purely **learned** aligners are fast, but may be **illegal** or **silently suboptimal**

The tension

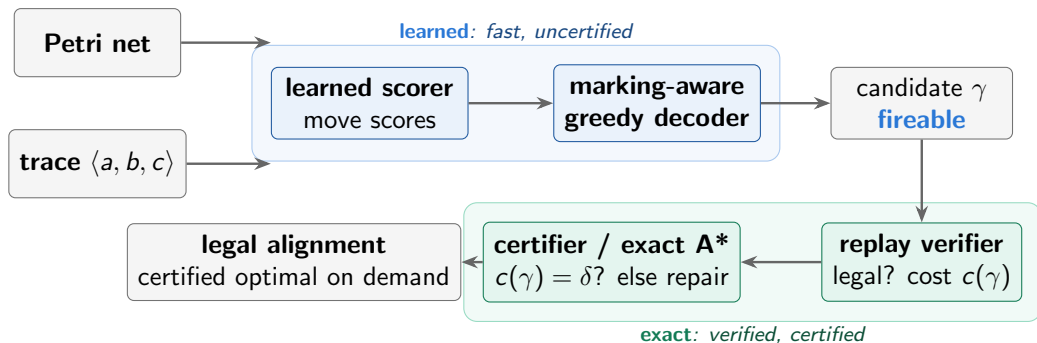
trust (exact, slow) vs. **tractability** (learned, fast)

Reusable candidates that stay semantically trustworthy

A small neural **candidate generator**, trained **once** on synthetic data, scores alignment moves for **any Petri net and any log**; replay and exact methods **verify, certify, or repair** every answer.

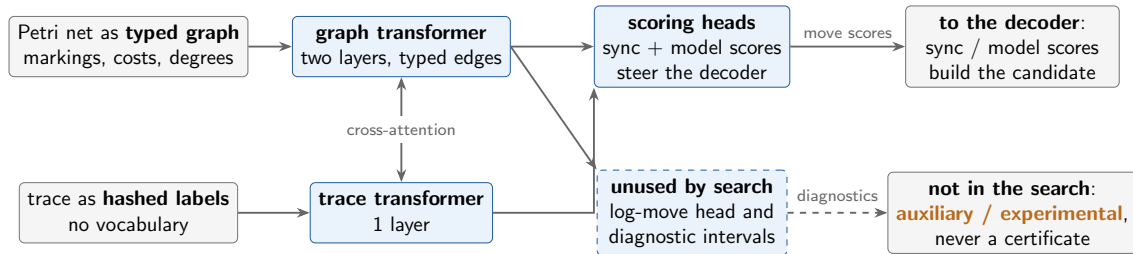
- One invariant: **neural output is never a certificate**
- Every answer is a **legal alignment with a known cost**; optionally a **certified optimal** one
- **Vocabulary-independent**: no retraining, no configuration per log

The Approach at a Glance



Degrades from **certified optimal** to a **safe upper bound**, **never to wrong**.

Architecture of the Learned Scorer



- Coupled encoders: **two graph layers** plus a **1-layer** trace transformer exchange messages; scores are **trace-dependent**
- The decoder consumes only **synchronous and model-move** scores; the **log-move head** and **diagnostic intervals** do not steer search
- Small model: about **1.7M** parameters, **two graph layers** ($d = 96$), runs on **CPU**

Training Loss: Imitating the Exact Aligner

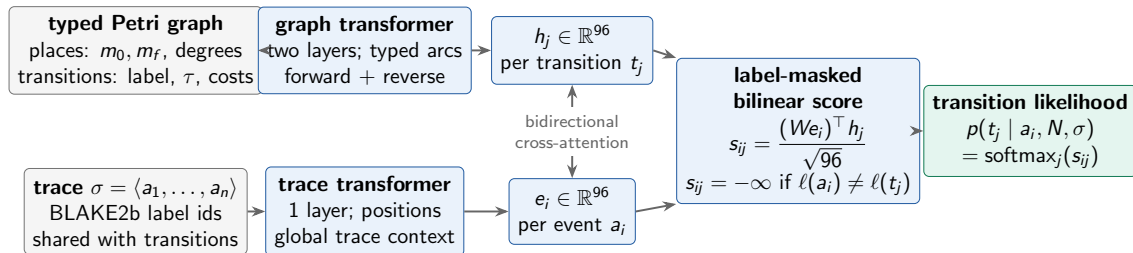
The model learns to **imitate** optimal alignments computed by exact A*:

$$\mathcal{L} = \underbrace{w_m (\mathcal{L}_{\text{sync}} + \mathcal{L}_{\text{log}} + \mathcal{L}_{\text{model}})}_{\text{imitation of moves, } w_m=1.0} + \underbrace{w_c \mathcal{L}_{\text{cost}} + w_b \mathcal{L}_{\text{bound}} + w_\ell \mathcal{L}_{\text{bal}} + w_e \mathcal{L}_{\text{ent}}}_{\text{unused by search, } w \leq 0.03}$$

- $\mathcal{L}_{\text{sync}}$: cross-entropy over transition **identities**, not labels: which concrete transition did the optimum fire? This is what teaches **duplicate-label** resolution
- $\mathcal{L}_{\text{log}} / \mathcal{L}_{\text{model}}$: which events did the optimum **skip**, which transitions did it fire **without a matching event**? Only model scores enter the decoder; $\mathcal{L}_{\text{log}}$ is **auxiliary**
- $\mathcal{L}_{\text{cost}}, \mathcal{L}_{\text{bound}}, \mathcal{L}_{\text{bal}}, \mathcal{L}_{\text{ent}}$ supervise experimental intervals and router regularization; they **never enter decoding, replay, or exact search**

Move imitation is dominant; diagnostics and the unused log-move head **do not** steer the candidate.

From Petri Net + Trace to Transition Likelihoods



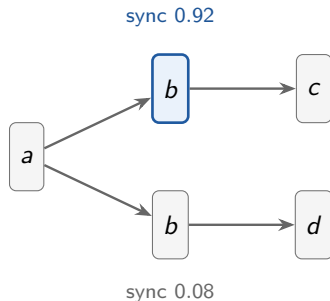
- The net encoder makes each h_j identify a **concrete transition** by initial/final markings, routing, and downstream context—not merely by its label.
- Cross-attention makes both h_j and e_i depend on the **whole net–trace pair**; the same transition can therefore score differently for different trace suffixes.
- The hard mask gives mismatched labels probability zero; among duplicates, the softmax ranks **transition identities**. During decoding, the current marking further restricts the choice to **enabled transitions**.

Where Learning Helps: Duplicate Labels

The decoder reads the trace **one event at a time**, always knowing the **current marking** of the Petri net:

- If exactly **one** enabled transition carries the event's label, the choice is **forced by the Petri net**: no learning needed
- If **several** enabled transitions carry the same label, the learned **synchronous score** decides which concrete transition the event refers to
- If **no** such transition is enabled, the **model-move scores** guide a short search that lets the Petri net catch up
- If even that fails, the event is a deviation: it becomes a **log move** (the log-move head is unused)

The candidate is always **replayable**; **optimality** is checked by the exact layer.



For the trace $\langle a, b, c \rangle$, both transitions labeled b are enabled; only the **upper** branch can continue with c , so the learned score selects it.

Training and Validation Data

Exact-labeled behavior families (imitation of the exact aligner):

- ① **draw a family**: one visible behavior, two Petri-net encodings, two shared traces (four rows); four equally weighted motifs (ordinary tree, duplicate/silent, concurrency, M pattern)
- ② **perturb** most traces with 1–3 deletions, insertions, substitutions, repetitions, swaps, or truncations (20% stay clean)
- ③ **label** with exact A^* : optimal alignment **with transition identities**, replay-verified

Split	Examples
Training	2,048
Validation	512
Test	512

Hashed labels $\Rightarrow$ the model learns **structure**, never vocabulary: one checkpoint for any log.

Corpus **2,048 / 512 / 512**. Best validation loss at **epoch 49 of 50**, about 41 s each, **single CPU**.

Evaluation: How

Protocol: all runs on a **single CPU**. LARA always in **fast mode**: the verified neural candidate is scored **as is**, never repaired, so no exact result contaminates the learned metrics. The **exact A* optimum** δ is the ground truth for every sample.

- **In-distribution quality:** 512 held-out rows from unseen behavior families; measures whether every candidate is **legal** (replayable), how often its cost **equals** the optimum δ , and **how far off** it is otherwise
- **Guidance ablation:** identical inputs, but the neural scores are replaced by fixed structural tie-breaks; isolates what **learning** adds over constrained search alone
- **Scaling and approximations:** random block-structured Petri nets never seen in training, plus PM4Py approximations on the same grids
- **Zero-shot real life:** two public logs (building permits, road traffic fines) with Petri nets discovered by the inductive miner; **unseen vocabularies, no fine-tuning**

Evaluation: Results

In-distribution ($n = 512$)	
legal alignments	100%
already optimal	91.0%
mean cost gap	0.18

Ablation: learning adds **+4.1** overall and **+26.6** on **duplicate-prefix** nets (95.3% vs. 68.8%); it also keeps quality **identifier-stable** at 84.4%.

- **Legality transfers everywhere:**
100% on unseen families and real logs
- Fast path stays at **4.6–8.7 ms**; at size 40, **30–44%** optimal, but faster than exact A*
- Receipt (invisible-heavy): **16.0 vs. 31.0 ms** and **6.7 vs. 12.7 ms**
- Established Petri-net approximations **generally return better candidates**

Why Consider LARA in Process Mining?

A reusable candidate plus replay and exact certification, not a replacement for exact or classical aligners.

learned proposal → **replay check** → **exact certificate or repair**

Portable first result

One vocabulary-independent checkpoint proposes a legal candidate with predictable latency; the fast path overtakes A* on harder tested models and the receipt nets.

Same conformance object

The output is still an event-by-event, transition-identity alignment with a replay-verified cost—usable by existing diagnostics, not an opaque fitness prediction.

Narrower learning gain

Learning pays on **duplicate prefixes** and **identifier-stable scoring**. Marking-aware search is already strong; classical approximations usually have better quality.

Practical role: a reusable proposal in front of replay and the exact aligner, **not a replacement for formal guarantees.**

What Can Be Improved

- Quality does **not transfer to large Petri nets** (30–44% optimal at size 40): classical approximations stay much closer to the optimum; broader training and stronger decoding are the next steps
- The decoder never compares the trained **log-move score** with catch-up paths, and it never revises an early commitment: joint ranking and beam search are the direct improvements
- Only **two real-life logs** tested, and certified mode still runs exact A^* on every trace: no end-to-end certified speedup is demonstrated

Takeaway

Learning proposes, exact reasoning certifies: the result is **never less trustworthy** than classical alignment.

- **Legality is unconditional:** every produced alignment can be replayed from the initial to the final marking, and this held in **100%** of the cases on synthetic families, on larger unseen Petri nets, and on real-life logs alike
- **91.0%** of the 512-row test split already carry the **exact optimal cost**; the other 9.0% are **repaired exactly**, never returned wrong. Certified mode still invokes A* on every trace
- Learning pays on **duplicate prefixes** (**95.3%** vs. 68.8%, +26.6) and keeps ordinary-tree quality **identifier-stable** at 84.4%; overall +4.1. Approximations generally beat this candidate quality
- The fast path stays at **4.6–8.7 ms** and overtakes exact A* on larger stress nets; on receipt it is **16.0 vs. 31.0 ms** and **6.7 vs. 12.7 ms**

Checkpoint, scripts, and results: <https://github.com/fit-alessandro-berti/lara-align>

Appendix

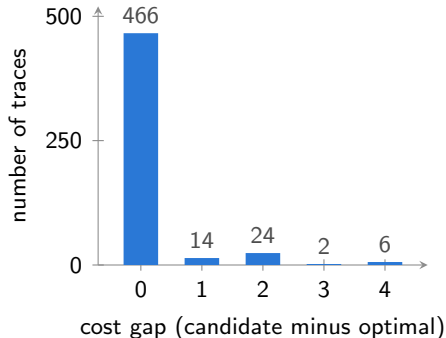
Detailed Evaluation Results

LARA always in fast mode; exact A* optimum as ground truth.

Appendix: In-Distribution Results

Held-out test split ($n = 512$)	
replayable (legal)	100.0%
already optimal / no repair	91.0%
exact transition-sequence match	61.1%
mean / median cost gap	0.18 / 0.00
p90 / p95 / max cost gap	0.0 / 2.0 / 4.0

466 of 512 candidates are optimal; 95% lie within two cost units, and **no gap exceeds four**. Sequence match is lower because several legal alignments can share the same cost.



Appendix: Guidance Ablation

Identical inputs and decoder; only the neural scores are replaced by fixed structural tie-breaks:

Metric ($n = 512$)	unguided	guided	Δ
replayable	100.0%	100.0%	0.0
equal to optimum cost	86.9%	91.0%	+4.1
exact transition-sequence match	57.8%	61.1%	+3.3
mean cost gap	0.26	0.18	-0.08
duplicate-prefix, optimal	68.8%	95.3%	+26.6
ordinary tree, isomorphic rename	76.6%	84.4%	+7.8

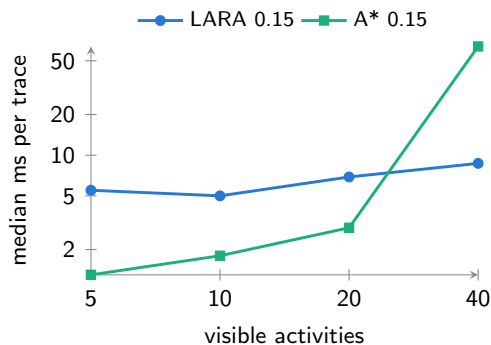
- On most motifs, marking-aware search already determines the candidate; guidance changes **no cost** for silent routing, concurrency, or either M representation
- Learning's documented gains are **duplicate-prefix** disambiguation and **identifier-stable** scoring (84.4% on both the canonical tree and its rename)

Appendix: Scaling Benchmark

Random block-structured Petri nets, **never seen in training** (10 samples per configuration, 30 s exact timeout):

size / dev.	legal	optimal	fast med.	exact med.
5 / 0.15	100%	90%	5.5 ms	1.3 ms
10 / 0.15	100%	80%	5.0 ms	1.8 ms
20 / 0.15	100%	60%	6.9 ms	2.9 ms
40 / 0.15	100%	30%	8.7 ms	63.9 ms
40 / 0.35	100%	44%	8.6 ms	24.7 ms

Fast path stays at **4.6–8.7 ms**; at size 40, **30–44%** optimal. **Legality is scale-invariant**. Crossover near **size 40** (dev. 0.15); first exact **timeout** appears there (dev. 0.35).



Appendix: Zero-Shot Results on Real-Life Logs

Same checkpoint, **no fine-tuning**; Petri nets discovered by the inductive miner (noise 0.0 = perfectly fitting, 0.5 = filtered):

Log	Noise	Legal	Optimal (trace-wtd.)	Max gap	Median LARA / exact
road traffic	0.0	100%	100.0%	0	5.4 / 1.9 ms
road traffic	0.5	100%	94.0%	3	5.2 / 1.6 ms
receipt	0.0	100%	82.6%	5	16.0 / 31.0 ms
receipt	0.5	100%	74.6%	7	6.7 / 12.7 ms

- **Legality transfers perfectly** to unseen vocabularies and invisible-heavy discovered models (47 invisible vs. 27 visible transitions on the fitting receipt net)
- Optimality is stronger when weighted by traces; misses concentrate in **rare, long variants** and remain safe upper bounds
- Guided and unguided quality coincide here: these nets have **no duplicate labels**, so the scorer's main ambiguity gain is not exercised

Appendix: Approximations Beat Current Candidate Quality

Independent replay of PM4Py approximations vs. LARA on the same held-out split and stress grid:

Method	Held-out opt.	Held-out gap	Stress opt.	Stress med.
LARA guided	91.0%	0.180	59.5%	7.69 ms
discounted A*	95.7%	0.090	75.9%	1.17 ms
tandem repeats	98.0%	0.039	93.6%	1.37 ms
sliding window	100.0%	0.000	100.0%	1.27 ms
fixed horizon	98.0%	0.047	82.9%	82.37 ms

- All tested Petri-net approximations beat LARA's exact-cost rate on held-out data; sliding window is perfect because every test trace fits one window of 20 events
- LARA's distinct advantage is **coverage** (legal in every case) plus a **reusable scorer** behind an explicit trust boundary
- The current decoder is **not** claimed to replace exact A* or these approximations

Appendix

Training Data Generation

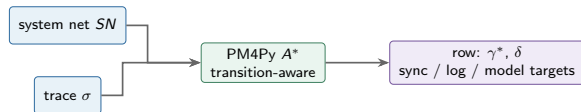
High-level tour of `paper/appendix.tex`: families, motifs, and the reference split.

Appendix: What One Training Row Contains

Every row is an **AlignmentSample**: one net, one trace, and the exact optimum for that pair.

- Petri net SN with m_0 and m_f
- observed trace σ
- exact alignment γ^* (transition identities)
- cost $\delta = c(\gamma^*)$
- generation metadata

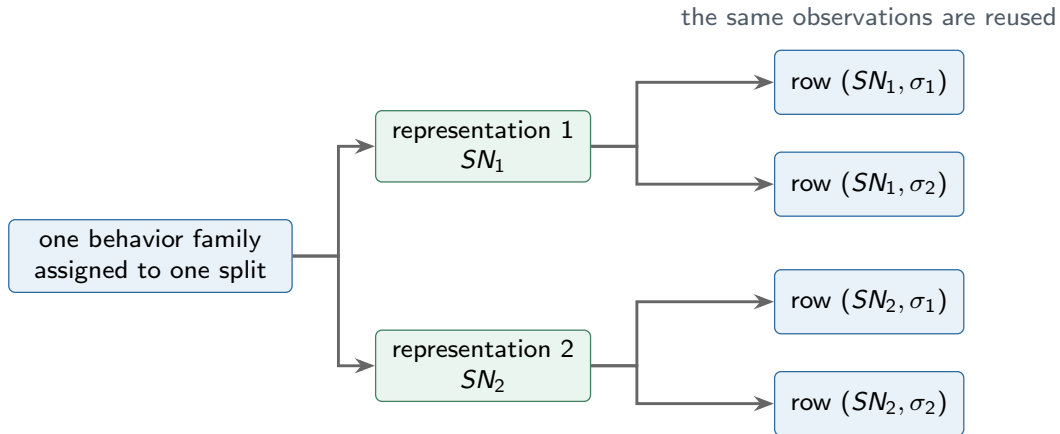
Unit costs: sync / silent = 0, visible log / model = 1.



Code path: `init_data.py` → families, exact, verify.

Appendix: A Behavior Family Expands to Four Rows

The split is chosen at **family** level: two Petri-net encodings of the same visible behavior share two observed traces.



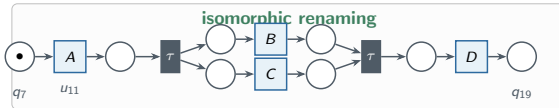
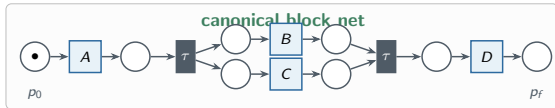
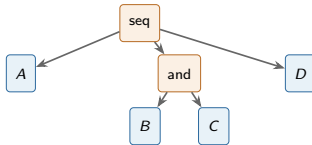
Appendix: Four Equally Weighted Motifs

motif	representation 1	representation 2
ordinary tree	canonical block net	isomorphic renaming
duplicate versus silent	duplicate-prefix net	silent-routing net
concurrent versus interleaved	parallel net	explicit interleaving net
M-pattern	canonical block net	non-free-choice M-pattern net

- A deterministic quota keeps every motif balanced inside each split
- Rejected attempts are replaced **without** changing the planned motif
- Controlled motifs get a shared suffix of two fresh activities

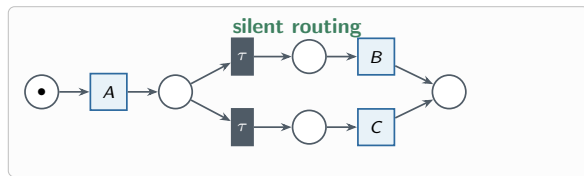
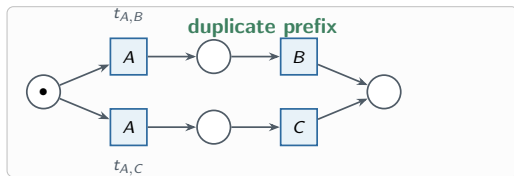
Appendix: Ordinary Tree — Canonical vs Rename

$\text{seq}(A, \text{and}(B, C), D)$: same language, new identifiers.



Appendix: Duplicate Prefix vs Silent Routing

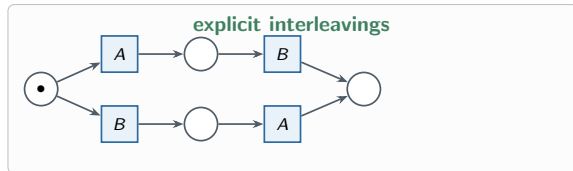
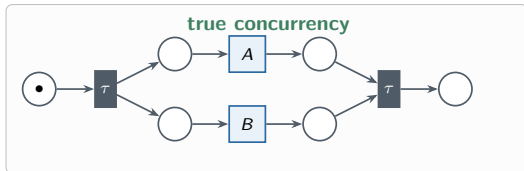
Same core language $\{AB, AC\}$; only the A identity differs.



both representations: $L_{\text{core}} = \{AB, AC\}$

Appendix: True Concurrency vs Explicit Interleaving

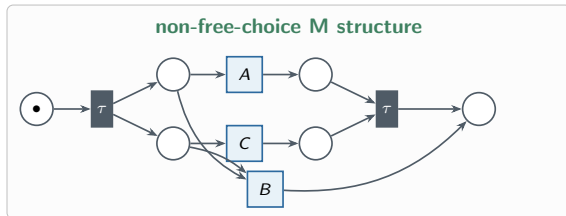
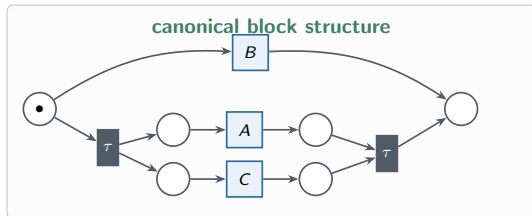
Same language $\{AB, BA\}$; different causal structure.



both representations: $L_{\text{core}} = \{AB, BA\}$

Appendix: M-pattern — Block vs Non-free-choice

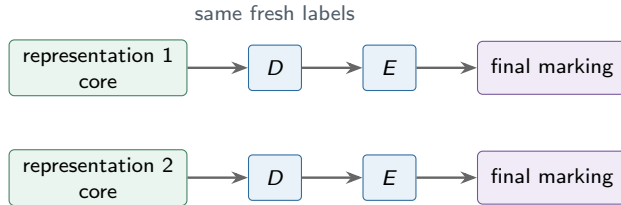
Same language $\{B, AC, CA\}$; the right net is not free-choice.



both representations: $L_{\text{core}} = \{B, AC, CA\}$

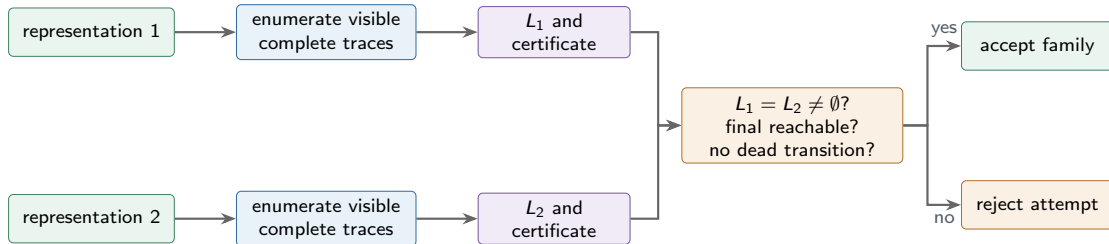
Appendix: Shared Suffix on Controlled Motifs

Both encodings of a controlled motif receive the same two fresh activities, so the comparison stays structural.



Appendix: Representations Must Match

Enumerate visible complete traces; reject empty, dead, or unequal languages (limits: 5,000 states, 10,000 traces, length 20).



Appendix: Reference Split Quotas

Seed 13; every motif stays inside its split; each family contributes four rows ($512 \times 4 = 2,048$ train).

